NAM Player 学習ガイド

JUCEで作る最小構成のNAM（Neural Amp Modeler）プレイヤーの、組み立てに必要なコードと、あとから追加できる機能をまとめた資料です。

空欄①〜⑦の答えはこの資料に含めていません。本体コードは空欄つきのまま載せています。

1. 全体の流れ

このプレイヤーの中では、次の2つの流れが別々に動いています。

(A) 読み込みの流れ（ボタンを押したときに1回）
ボタン → ファイル選択 → `loadModel()` → `nam::get_dsp()` でモデルを作る → `Reset()` で再生条件に合わせる → メンバ変数 `model` に入れる

(B) 音声処理の流れ（1秒間に何百回も）
オーディオインターフェースの入力 → `processBlock()` → 左チャンネルを取り出す → `model->process()` で変換 → 結果を左右に書き戻す → 出力

(A)と(B)は別々のスレッド（同時に並んで動く処理の流れ）で動きます。そのため、(A)がモデルを差し替えている最中に(B)が触らないよう、`modelLock`（SpinLock）で鍵をかけています。

2. ファイル構成と空欄の一覧

空欄	ファイル	何のための行か
①	PluginProcessor.h	読み込んだモデルを持ち続ける変数
②	PluginProcessor.cpp	ファイルからモデルを作る
③	PluginProcessor.cpp	新しいモデルを今のサンプルレートに合わせる
④	PluginProcessor.cpp	入力の音（左チャンネル）の場所を取り出す
⑤	PluginProcessor.cpp	NAMに音を変換させる
⑥	PluginProcessor.cpp	変換結果を各チャンネルに書き戻す
⑦	PluginEditor.cpp	選んだファイルを本体に渡す

3. ビルド手順

空欄をすべて埋めたあとに実行します。JUCEとNAM Coreは初回のcmake時に自動でダウンロードされます。

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release --target NamPlayer_Standalone
```

必要なもの: CMake 3.22以上、C++20対応のコンパイラ（Windows: Visual Studio 2022 / macOS: Xcode）。Linuxでは加えて ALSA・X11・freetype などの開発用パッケージが要ります。

できあがるアプリ: `build/NamPlayer_artefacts/Release/Standalone/` の中。

4. 自分で調べるときの手順（関数の戻り値を知る）

関数が何を返すかは、ライブラリのヘッダファイル（.h、関数の「説明書き」だけが並んだファイル）に書いてあります。NAM Coreなら初回ビルド後、`build/_deps/namcore-src/NAM/get_dsp.h` にあります。

```
std::unique_ptr<DSP> get_dsp(const std::filesystem::path config_filename, ...);
^^^^^^^^^^^^^^^^^^^^ ^^^^^^^  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  戻り値の型          関数名   受け取るもの（引数）
```

関数宣言は「戻り値の型 → 関数名 →（引数）」の順に並びます。ここでは `DSP` に `nam::` が付いていませんが、これはこの宣言自体が `namespace nam { ... }` の中に書かれているからです。外から使うときは `nam::DSP` と書きます。

探し方は3つあります。(1) IDEで関数名を右クリックし「定義へ移動」（Visual Studio: F12 / Xcode: Command+クリック）。(2) ヘッダファイルを開いて関数名で検索。(3) ライブラリ付属のサンプル（NAM Coreなら `tools/loadmodel.cpp`）で使い方を見る。

5. 全コード（空欄つき）

CMakeLists.txt

```
# ============================================================
#  NAM Player - ビルド設定（ここは演習の対象外。触らなくてOK）
# ============================================================
cmake_minimum_required(VERSION 3.22)
project(NamPlayer VERSION 0.1.0)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

include(FetchContent)

# JUCE本体をネットから取ってくる
FetchContent_Declare(JUCE
    GIT_REPOSITORY https://github.com/juce-framework/JUCE.git
    GIT_TAG        8.0.10
    GIT_SHALLOW    TRUE)
FetchContent_MakeAvailable(JUCE)

# NAMの推論ライブラリ（.namファイルを読んで音を変換する部分）
FetchContent_Declare(NAMCore
    GIT_REPOSITORY https://github.com/sdatkinson/NeuralAmpModelerCore.git
    GIT_TAG        d3189ed482ded880c128bf312941266c4fa5ceec
    GIT_SUBMODULES Dependencies/eigen)
FetchContent_Populate(NAMCore)   # ソースだけ取得（NAM側のCMakeは使わない）

file(GLOB NAM_SOURCES
    ${namcore_SOURCE_DIR}/NAM/*.cpp
    ${namcore_SOURCE_DIR}/NAM/*/*.cpp)

juce_add_plugin(NamPlayer
    COMPANY_NAME      "MyName"
    PLUGIN_MANUFACTURER_CODE Mynm
    PLUGIN_CODE       Namp
    FORMATS           Standalone VST3
    PRODUCT_NAME      "NAM Player")

target_sources(NamPlayer PRIVATE
    Source/PluginProcessor.cpp
    Source/PluginEditor.cpp
    ${NAM_SOURCES})

target_include_directories(NamPlayer PRIVATE
    ${namcore_SOURCE_DIR}
    ${namcore_SOURCE_DIR}/NAM
    ${namcore_SOURCE_DIR}/Dependencies/eigen
    ${namcore_SOURCE_DIR}/Dependencies/nlohmann)

target_compile_definitions(NamPlayer PRIVATE
    NAM_SAMPLE_FLOAT=1            # NAMにfloatで計算させる（JUCEのバッファと同じ型）
    JUCE_WEB_BROWSER=0
    JUCE_USE_CURL=0
    JUCE_VST3_CAN_REPLACE_VST2=0)

target_link_libraries(NamPlayer PRIVATE
    juce::juce_audio_utils
    PUBLIC
    juce::juce_recommended_config_flags
    juce::juce_recommended_warning_flags)
```

Source/PluginProcessor.h

```
#pragma once

#include <juce_audio_utils/juce_audio_utils.h>
#include "NAM/dsp.h"

// ============================================================
//  音の処理をする本体。DAWやStandaloneアプリから呼ばれる。
// ============================================================
class NamPlayerProcessor : public juce::AudioProcessor
{
public:
```

```
    NamPlayerProcessor();

    // 再生が始まる直前に1回呼ばれる（サンプルレートなどが決まるタイミング）
    void prepareToPlay (double sampleRate, int samplesPerBlock) override;
    void releaseResources() override {}

    // 音の小さなかたまり（ブロック）ごとに何度も呼ばれる
    void processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer&) override;

    // .namファイルを読み込む。成功したら true を返す
    bool loadModel (const juce::File& file);

    // ---- 以下はJUCEが要求する決まり文句（演習の対象外） ----
    juce::AudioProcessorEditor* createEditor() override;
    bool hasEditor() const override { return true; }
    const juce::String getName() const override { return "NAM Player"; }
    bool acceptsMidi() const override { return false; }
    bool producesMidi() const override { return false; }
    double getTailLengthSeconds() const override { return 0.0; }
    int getNumPrograms() override { return 1; }
    int getCurrentProgram() override { return 0; }
    void setCurrentProgram (int) override {}
    const juce::String getProgramName (int) override { return {}; }
    void changeProgramName (int, const juce::String&) override {}
    void getStateInformation (juce::MemoryBlock&) override {}
    void setStateInformation (const void*, int) override {}

private:
    // 読み込んだアンプモデル（まだ無いときは空っぽ = nullptr）
    /* ____ 空欄① ____ */

    // モデルの差し替え中に音声処理が触らないようにする鍵
    juce::SpinLock modelLock;

    // NAMの出力を一旦受け取る置き場
    std::vector<float> outputBuffer;

    double currentSampleRate = 48000.0;
    int currentBlockSize = 512;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (NamPlayerProcessor)
};
```

Source/PluginProcessor.cpp

```
#include "PluginProcessor.h"
#include "PluginEditor.h"
#include "NAM/get_dsp.h"

NamPlayerProcessor::NamPlayerProcessor()
    : AudioProcessor (BusesProperties()
                        .withInput  ("Input",  juce::AudioChannelSet::stereo(), true)
                        .withOutput ("Output", juce::AudioChannelSet::stereo(), true))
{
}

// ------------------------------------------------------------
//  .namファイルの読み込み（ボタンを押したときに呼ばれる）
// ------------------------------------------------------------
bool NamPlayerProcessor::loadModel (const juce::File& file)
{
    // JUCEのファイルを、NAMが受け取れる形のパスに変換
    const std::filesystem::path path (file.getFullPathName().toWideCharPointer());

    std::unique_ptr<nam::DSP> newModel;
    try
    {
        newModel = /* ____ 空欄② ____ */;
    }
    catch (const std::exception&)
    {
        return false; // 壊れたファイルなど
    }

    if (newModel == nullptr)
        return false;
```

```
    // 今の再生条件に合わせて準備させる
    /* ____ 空欄③ ____ */

    // 鍵をかけてから、古いモデルと新しいモデルを入れ替える
    {
        const juce::SpinLock::ScopedLockType lock (modelLock);
        std::swap (model, newModel);
    }
    return true; // ここで newModel(=古いモデル) が自動で片付けられる
}

void NamPlayerProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
{
    currentSampleRate = sampleRate;
    currentBlockSize = samplesPerBlock;
    outputBuffer.resize ((size_t) samplesPerBlock);

    const juce::SpinLock::ScopedLockType lock (modelLock);
    if (model != nullptr)
        model->Reset (currentSampleRate, currentBlockSize);
}

// ------------------------------------------------------------
//  音声処理の本体
// ------------------------------------------------------------
void NamPlayerProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer&)
{
    juce::ScopedNoDenormals noDenormals;
    const int numSamples = buffer.getNumSamples();

    // 鍵が取れない（差し替え中）か、モデルが無いなら、何もせず素通し
    const juce::SpinLock::ScopedTryLockType lock (modelLock);
    if (! lock.isLocked() || model == nullptr || numSamples > (int) outputBuffer.size())
        return;

    // ギターは1本 = 左チャンネル(0番)だけを入力として使う
    float* input = /* ____ 空欄④ ____ */;
    float* output = outputBuffer.data();

    // NAMに変換してもらう（input → output）
    /* ____ 空欄⑤ ____ */

    // 変換結果を、左右すべてのチャンネルに書き戻す
    for (int ch = 0; ch < buffer.getNumChannels(); ++ch)
        /* ____ 空欄⑥ ____ */
}

juce::AudioProcessorEditor* NamPlayerProcessor::createEditor()
{
    return new NamPlayerEditor (*this);
}

// JUCEがプラグインを作るときに呼ぶ入口（決まり文句）
juce::AudioProcessor* JUCE_CALLTYPE createPluginFilter()
{
    return new NamPlayerProcessor();
}
```

Source/PluginEditor.h

```
#pragma once

#include "PluginProcessor.h"

// ============================================================
//  画面。ボタン1つと、文字表示1つだけ。
// ============================================================
class NamPlayerEditor : public juce::AudioProcessorEditor
{
public:
    explicit NamPlayerEditor (NamPlayerProcessor&);

    void paint (juce::Graphics&) override;
    void resized() override;

private:
```

```
    NamPlayerProcessor& processor;

    juce::TextButton loadButton { "Load .nam" };
    juce::Label statusLabel;
    std::unique_ptr<juce::FileChooser> chooser;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (NamPlayerEditor)
};
```

Source/PluginEditor.cpp

```
#include "PluginEditor.h"

NamPlayerEditor::NamPlayerEditor (NamPlayerProcessor& p)
    : AudioProcessorEditor (&p), processor (p)
{
    statusLabel.setText ("No model loaded", juce::dontSendNotification);
    statusLabel.setJustificationType (juce::Justification::centred);

    addAndMakeVisible (loadButton);
    addAndMakeVisible (statusLabel);

    // ボタンが押されたら、ファイル選択ダイアログを開く
    loadButton.onClick = [this]
    {
        chooser = std::make_unique<juce::FileChooser> ("Select a .nam file", juce::File{}, "*.nam");

        chooser->launchAsync (juce::FileBrowserComponent::openMode | juce::FileBrowserComponent::canSelectFiles,
                              [this] (const juce::FileChooser& fc)
                              {
                                  const juce::File file = fc.getResult();
                                  if (file == juce::File{})
                                      return; // キャンセルされた

                                  const bool ok = /* ____ 空欄⑦ ____ */;

                                  statusLabel.setText (ok ? file.getFileName() : "Failed to load",
                                                       juce::dontSendNotification);
                              });
    };

    setSize (300, 120);
}

void NamPlayerEditor::paint (juce::Graphics& g)
{
    g.fillAll (juce::Colours::darkslategrey);
}

void NamPlayerEditor::resized()
{
    auto area = getLocalBounds().reduced (10);
    loadButton.setBounds (area.removeFromTop (40));
    statusLabel.setBounds (area);
}
```

6. 追加できる機能

どれも空欄をすべて埋めて動くようになってから取り組む前提です。下のコードは本体に足した状態でビルドが通ることを確認済みです。

6-1. 入力ゲイン・出力ゲインのつまみ

何のためか: NAMモデルは、学習したときの入力の大きさで最もそれらしく鳴ります。ギターの出力はギターによって違うので、モデルに入る前の音量（入力ゲイン）を調整できるようにします。モデルごとに出てくる音量も違うため、出力ゲインもあると便利です。

使う部品: AudioProcessorValueTreeState（つまみの値をまとめて管理し、DAWからの自動操作や保存にも対応させる仕組み）。

PluginProcessor.h の public: に追加

```
static juce::AudioProcessorValueTreeState::ParameterLayout createLayout();
juce::AudioProcessorValueTreeState params { *this, nullptr, "PARAMS", createLayout() };
```

PluginProcessor.cpp に追加（つまみの定義。-24dB〜+24dB、初期値0dB）

```
juce::AudioProcessorValueTreeState::ParameterLayout NamPlayerProcessor::createLayout()
{
    juce::AudioProcessorValueTreeState::ParameterLayout layout;
    layout.add (std::make_unique<juce::AudioParameterFloat> (juce::ParameterID { "input", 1 },  "Input",  -24.0f, 24.0f, 0.0f));
    layout.add (std::make_unique<juce::AudioParameterFloat> (juce::ParameterID { "output", 1 }, "Output", -24.0f, 24.0f, 0.0f));
    return layout;
}
```

processBlock の中（モデルを通す前に入力ゲイン、最後に出力ゲイン）

```
const float inGain  = juce::Decibels::decibelsToGain (params.getRawParameterValue ("input")->load());
const float outGain = juce::Decibels::decibelsToGain (params.getRawParameterValue ("output")->load());
buffer.applyGain (0, 0, numSamples, inGain);   // 空欄④の前に置く
// ...（モデルの処理）...
buffer.applyGain (outGain);                    // 関数の最後に置く
```

PluginEditor.h のメンバ（Attachment がつまみと値を自動でつなぐ。resized() で setBounds、コンストラクタで addAndMakeVisible も必要）

```
juce::Slider inputSlider;
juce::AudioProcessorValueTreeState::SliderAttachment inputAttach { processor.params, "input", inputSlider };
```

6-2. キャビネットIRの読み込み

何のためか: 多くのNAMモデルはアンプ部分だけを学習していて、スピーカーキャビネットの音は含みません。そのまま鳴らすとジャリジャリした音になるので、キャビネットの響きを録った音声ファイル（IR = インパルス応答）を後ろにかけます。

使う部品: juce::dsp::Convolution（IRの響きを音に重ねる処理）。CMakeLists.txt の target_link_libraries に juce::juce_dsp を、PluginProcessor.h に #include <juce_dsp/juce_dsp.h> を追加します。

```
// PluginProcessor.h の private:
juce::dsp::Convolution cabinet;

// prepareToPlay の中
cabinet.prepare ({ sampleRate, (juce::uint32) samplesPerBlock, (juce::uint32) getTotalNumOutputChannels() });

// IRファイルを読む関数（ボタンから呼ぶ。読み込み処理はJUCEが裏で安全に行う）
void NamPlayerProcessor::loadIR (const juce::File& file)
{
    cabinet.loadImpulseResponse (file, juce::dsp::Convolution::Stereo::no,
                                 juce::dsp::Convolution::Trim::yes, 0,
                                 juce::dsp::Convolution::Normalise::yes);
}

// processBlock の中、空欄⑥のループの後
juce::dsp::AudioBlock<float> block (buffer);
cabinet.process (juce::dsp::ProcessContextReplacing<float> (block));
```

6-3. 最後に読み込んだモデルを覚えておく

何のためか: 今のままだと、DAWのプロジェクトを開き直すたびにモデルを選び直す必要があります。DAWは保存時に `getStateInformation`、読み込み時に `setStateInformation` を呼ぶので、そこでファイルの場所を書き出し・読み戻しします。

```
// PluginProcessor.h: 2つの関数の {} を外して宣言だけにし、private: に追加
juce::File lastModelFile;   // loadModel が成功したら lastModelFile = file; を入れる

// PluginProcessor.cpp
void NamPlayerProcessor::getStateInformation (juce::MemoryBlock& dest)
{
    juce::ValueTree state ("STATE");
    state.setProperty ("modelPath", lastModelFile.getFullPathName(), nullptr);
    juce::MemoryOutputStream out (dest, false);
    state.writeToStream (out);
}

void NamPlayerProcessor::setStateInformation (const void* data, int size)
{
    auto state = juce::ValueTree::readFromData (data, (size_t) size);
    const juce::File file (state.getProperty ("modelPath").toString());
    if (file.existsAsFile())
        loadModel (file);
}
```

6-1と組み合わせる場合は、つまみの値（`params.copyState()`）も同じ `ValueTree` に入れて保存します。